

MIP_PROMPT_LIBRARY_V2_PART3_METADATA.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 3 - Metadata, Semantic Models, and System Intelligence Prompts

Version: 2.1

Status: Enhanced Enterprise Architecture Edition

Purpose

Metadata is not merely a representation of a legacy system.

Metadata is the foundation for system intelligence, resilience engineering, modernization planning, operational understanding, governance, AI reasoning, and autonomous analysis.

Source code represents implementation details.

Metadata represents system knowledge.

Semantic metadata represents business intent.

Behavioral metadata represents runtime reality.

Relationship metadata represents system connectivity.

Operational metadata represents resilience characteristics.

MIP transforms:

Source Code

↓

Structural Metadata

↓

Behavioral Metadata

↓

Semantic Metadata



Relationship Metadata



Knowledge Graph



Reasoning Engine



Modernization Intelligence



Autonomous System Understanding

Without comprehensive metadata, modernization efforts remain incomplete, fragile, and dependent on tribal knowledge.

Metadata Design Principles

Evidence Based

Every metadata element must reference source evidence.

Evidence sources may include:

- COBOL
- JCL
- Copybooks
- DB2
- VSAM
- IMS
- CICS
- MQ
- Scheduler Definitions
- Runtime Logs
- SMF Records
- Operational Documentation

Canonical

One representation regardless of source technology.

The same business concept should map to a single canonical model.

Traceable

Every entity must be traceable back to source artifacts and runtime evidence.

Graph Ready

Metadata must support graph traversal and graph analytics.

Modernization Ready

Metadata should support migration, decomposition, API generation, cloud transformation, and service extraction.

Runtime Aware

Static analysis alone is insufficient.

Metadata should incorporate runtime observations whenever available.

Explainable

Every recommendation generated by MIP must be explainable through metadata lineage.

Resilience Focused

Metadata should capture operational dependencies, failure points, recovery paths, and critical business assets.

Advanced Metadata Acquisition Strategies

Traditional parsers provide only partial understanding.

MIP should support multiple metadata acquisition methodologies.

Static Analysis

Extract metadata from:

- COBOL
- JCL
- Copybooks
- SQL
- DB2 DDL
- VSAM Definitions
- IMS Definitions

Purpose:

Build structural understanding.

Runtime Analysis

Extract metadata from:

- SMF Records
- Job Logs
- Execution Statistics
- Scheduler History
- Transaction Logs

Purpose:

Understand actual system behavior.

Dynamic Dependency Discovery

Identify:

- Frequently executed paths
- Runtime call chains
- Dataset usage patterns
- Transaction flows

Purpose:

Reveal dependencies not visible through static analysis.

AI-Assisted Semantic Discovery

Use LLMs and reasoning models to infer:

- Business capabilities
- Functional domains
- Service boundaries
- Business processes
- Candidate APIs

Purpose:

Bridge technical and business understanding.

Graph-Based Pattern Mining

Apply graph algorithms to discover:

- Hidden clusters
- Application boundaries
- Circular dependencies
- Critical hubs
- High-risk components

Algorithms:

- PageRank
- Community Detection
- Louvain Clustering
- Betweenness Centrality
- Connected Components
- Dependency Density Analysis

Purpose:

Reveal architecture automatically.

Embedding-Based Similarity Analysis

Generate embeddings from:

- Source code
- Metadata
- Documentation

Identify:

- Similar programs

- Duplicate logic
- Reusable services
- Modernization candidates

Purpose:

Enable AI-driven discovery.

Event and Data Flow Reconstruction

Reconstruct:

- Batch chains
- Data pipelines
- Transaction journeys
- Business workflows

Purpose:

Understand end-to-end system behavior.

Prompt 11

Program Metadata Model

File

prompts/metadata/program_entity.md

Purpose

Design the canonical Program entity used throughout MIP.

Context

Programs are primary execution units.

Programs often become future services, APIs, or modernization units.

The Program entity becomes one of the most connected nodes in the platform.

Inputs

- COBOL Metadata
- Runtime Metadata
- Call Graph Metadata
- Operational Metadata

Instructions

Design a canonical Program entity.

Include:

Identity

- Program Name
- Program Type
- Source Location
- Language
- Version
- Repository

Dependencies

- Called Programs
- Calling Programs
- Copybooks
- Tables
- Files
- MQ Queues
- CICS Transactions
- IMS Resources

Execution

- Entry Points
- Batch Usage
- Online Usage
- Scheduler Usage
- Runtime Frequency

Operational Characteristics

- Criticality
- Availability Requirements
- Recovery Dependencies
- Failure Impact

Metrics

- Lines of Code
- Complexity
- Dependency Count
- Change Frequency
- Runtime Frequency

AI-Derived Metadata

- Business Capability

- Functional Domain
- Service Candidate Score
- API Candidate Score

Generate:

- Pydantic Model
- SQLite Mapping
- Graph Mapping
- Vector Embedding Strategy

Expected Output

Program Entity Specification

Validation Rules

Versioning Strategy

Modernization Attributes

Constraints

Must support millions of relationships.

Success Criteria

Program entity supports graph analytics, AI reasoning, modernization planning, and resilience analysis.

Prompt 12

Job Metadata Model

File

prompts/metadata/job_entity.md

Purpose

Create canonical representation of batch jobs.

Context

Batch jobs are often true system entry points and business process orchestrators.

Inputs

- JCL Metadata
- Scheduler Metadata

- Runtime Metadata

Instructions

Design Job entity.

Include:

Identity

- Job Name
- Job Class
- Schedule
- Owner

Execution

- Steps
- Programs
- Procedures
- Utilities

Data Dependencies

- Input Datasets
- Output Datasets
- Temporary Datasets

Operational Metadata

- Runtime Duration
- Frequency
- SLA
- Failure History

Business Metadata

- Business Process
- Criticality
- Regulatory Impact

Generate:

- Metadata Model
- Graph Representation
- Lineage Mapping
- Execution Flow Model

Expected Output

Job Entity Definition

Relationship Model

Validation Rules

Operational Metadata Model

Constraints

Support large enterprise batch ecosystems.

Success Criteria

Job execution flows become graph-ready and operationally explainable.

Prompt 13

Dataset Metadata Model

File

prompts/metadata/dataset_entity.md

Purpose

Represent enterprise datasets consistently.

Context

Datasets connect jobs, programs, applications, and business processes.

Datasets are often the hidden backbone of legacy systems.

Inputs

- JCL Metadata
- COBOL Metadata
- Runtime Metadata

Instructions

Model:

Identity

- Dataset Name
- Dataset Type
- Storage Class

Ownership

- Application

- Business Domain
- Steward

Usage

- Producers
- Consumers
- Access Frequency

Governance

- Retention
- Classification
- Compliance Requirements

Operational Metadata

- Availability Requirements
- Backup Strategy
- Recovery Dependencies

Generate lineage-ready metadata.

Expected Output

Dataset Entity

Lineage Relationships

Governance Metadata

Operational Metadata

Success Criteria

Complete data lineage and operational dependency analysis can be reconstructed.

Prompt 14

DB2 Metadata Model

File

prompts/metadata/db2_entity.md

Purpose

Represent database dependencies consistently.

Context

DB2 objects frequently become future service boundaries, APIs, and domain models.

Inputs

- DB2 Metadata
- SQL Metadata
- Runtime Access Metadata

Instructions

Design metadata for:

Database Objects

- Tables
- Views
- Aliases
- Indexes
- Stored Procedures
- Packages

Usage Metadata

Capture:

- CRUD Usage
- Access Frequency
- Join Patterns
- Transaction Participation

Modernization Metadata

Capture:

- Domain Ownership
- Service Boundary Candidates
- Data Product Candidates

Operational Metadata

Capture:

- Criticality
- Availability Requirements
- Recovery Dependencies

Expected Output

DB2 Entity Model

Relationship Definitions

Migration Metadata

Operational Metadata

Success Criteria

Database dependencies support modernization, decomposition, and resilience analysis.

Prompt 15

Relationship Metadata Model

File

prompts/metadata/relationship_model.md

Purpose

Create canonical relationship taxonomy.

Context

Relationships power the knowledge graph and system reasoning engine.

Inputs

All metadata entities.

Instructions

Design relationship types.

Core Structural Relationships

- CALLS
- EXECUTES
- USES
- READS
- WRITES
- CONTAINS
- DEPENDS_ON
- PRODUCES
- CONSUMES

Operational Relationships

- FAILS_WITH
- RECOVERS_WITH
- BACKS_UP
- RESTARTS_FROM

Business Relationships

- SUPPORTS_PROCESS
- SUPPORTS_CAPABILITY
- BELONGS_TO_DOMAIN

Modernization Relationships

- SERVICE_CANDIDATE
- API_CANDIDATE
- MIGRATION_GROUP

Runtime Relationships

- FREQUENTLY_EXECUTES_WITH
- FREQUENTLY_ACCESSES
- RUNTIME_DEPENDS_ON

Generate:

- Taxonomy
- Cardinality Rules
- Validation Rules
- Confidence Scoring Model

Expected Output

Relationship Catalog

Schema Definition

Graph Mapping

Confidence Framework

Success Criteria

Every dependency, behavior, and business relationship can be represented consistently.

Prompt 15A

Business Capability Metadata Model

File

prompts/metadata/business_capability_entity.md

Purpose

Create metadata representing business capabilities.

Context

Modernization should align with business outcomes rather than technical structures.

Inputs

- Program Metadata
- Job Metadata
- Documentation
- AI Semantic Analysis

Instructions

Identify:

- Business Capabilities
- Business Functions
- Domains
- Processes

Map technical assets to business capabilities.

Expected Output

Business Capability Model

Capability Relationships

Domain Mapping

Success Criteria

Technical assets become understandable in business terms.

Prompt 15B

Application Boundary Discovery Metadata

File

prompts/metadata/application_boundary_model.md

Purpose

Automatically identify logical applications.

Context

Many enterprises lack accurate application inventories.

Inputs

- Graph Metadata
- Runtime Metadata
- Dependency Metadata

Instructions

Apply:

- Community Detection
- Dependency Clustering
- Graph Partitioning

Generate:

- Application Boundaries
- Dependency Scores
- Ownership Candidates

Expected Output

Application Metadata Model

Boundary Definitions

Confidence Scores

Success Criteria

Logical applications emerge automatically from metadata.

Metadata Completion Criteria

MIP should be able to answer:

Structural Questions

- What is a Program?
- What is a Job?
- What is a Dataset?
- What is a Table?
- How are they related?

Behavioral Questions

- How does the system actually execute?
- Which paths are most frequently used?
- Which components are rarely used?

Operational Questions

- What fails if this component fails?
- What are the recovery paths?
- Which assets are mission critical?

Business Questions

- Which business capability does this support?
- Which applications belong together?
- What business process is impacted?

Modernization Questions

- What should become a microservice?
- What should become an API?
- What can be retired?
- What can be migrated together?

AI and Graph Questions

- Which components are central to the architecture?
- Which dependencies are hidden?
- Which clusters represent applications?
- Which assets represent modernization opportunities?

Only after metadata is complete should graph construction begin.

End of Part 3.